

Bearer Token APIs

Protect your Laravel API with Sanctum tokens.

Learning Goals

- Explain bearer tokens, the login → token → request flow, and why the API is stateless.
- Distinguish authentication from authorization, and Sanctum tokens from JWT.
- Add login, `auth:sanctum` route protection, and `me` and `logout` actions.

In the previous lesson, anyone could `POST` , `PUT` , or `DELETE` a student. The API had no idea **who** was calling.

- In this section, we add **authentication**: the client proves who it is, and the API remembers nothing but a token.
- In Module 1 (`7_Security/bearer`), you wrote this by hand in PHP.
- Here, Laravel and **Sanctum** do most of the work.
- We keep the same `studentdb` database and the same `students` table.

From Module 1 to Laravel

Idea	Module 1 (PHP)	This lesson (Laravel)
Read the header	<code>getBearerToken()</code> with <code>preg_match</code>	Built into Sanctum
Check the token	<code>isValidToken()</code> with a hard-coded array	<code>personal_access_token</code> table
Create a token	Fixed demo tokens	<code>\$user->createToken()</code>
Require login	<code>requireAuth()</code>	- >middleware('auth:sanctum')
Error response	<code>sendJsonError(401, ...)</code>	Automatic <code>401</code>

What Is a Bearer Token?

A **bearer token** is a secret string. Whoever *bears* (carries) it is treated as the logged-in user.

- Think of a hotel key card: the door does not ask who you are, it only checks the card.
- The client sends the token in an HTTP header:

```
Authorization: Bearer 1|Xk3v9...
```

- **Bearer** is the scheme name, followed by a space and the token.

The Flow

```
1. POST /api/login    email + password  
                      <- 200 {"token": "1|Xk3v9..."}  
  
2. GET  /api/me       Authorization: Bearer 1|Xk3v9...  
                      <- 200 {"user": {...}}  
  
3. GET  /api/me       (no token)  
                      <- 401 Unauthenticated
```

- The password travels **once**, at login; After that, only the token travels.
- The API is **stateless**: no session is kept; Each request carries its own proof.

Anyone who steals a bearer token can use it.

- Always use **HTTPS** in production.
- Give tokens an **expiration** time.
- Never commit a token to Git or print it in logs.

Authentication vs. Authorization

	Question	Example
Authentication	Who are you?	A valid token belongs to <code>student@university.edu</code>
Authorization	What may you do?	Only a teacher may delete a student

- Sanctum handles authentication. Your code must still decide authorization.
- This lesson only requires "a logged-in user" for write actions.

What Is Sanctum?

Laravel Sanctum is Laravel's official, lightweight package for API authentication.

- It issues **personal access tokens** and checks them on each request.
- It also supports cookie login for single-page apps. We do not use that here.
- Tokens are random strings stored in the database, so the server can **revoke** one at any time by deleting its row.

Sanctum Token vs. JWT

	Sanctum token	JWT (Module 1 <code>jwt_token</code>)
What is it?	Random string	Signed data that contains the user
Checked by	Database lookup	Signature check, no lookup
Revoke early	Easy: delete the row	Hard: it is valid until it expires
Best for	Your own API and apps	Tokens passed between services

Sanctum Setup

The script `run1-5.sh` runs this for you:

```
php artisan install:api
```

- Creates `routes/api.php` and `config/sanctum.php`.
- Adds the `personal_access_tokens` migration **file**. It does not create the table.

`install:api` was already used in `run1-4.sh`, but the token table was not needed there. Lesson 5-2 shows how the table is created.

The `personal_access_tokens` Table

Column	Meaning
<code>tokenable_type</code> , <code>tokenable_id</code>	Which user owns the token
<code>name</code>	A label you choose, such as <code>api-token</code>
<code>token</code>	A SHA-256 hash of the token, not the token itself
<code>abilities</code>	What the token may do (<code>["*"]</code> means everything)
<code>last_used_at</code> , <code>expires_at</code>	Usage and expiration times

Let the User Own Tokens

Laravel's default `User` model does not include Sanctum's `HasApiTokens` trait.

- `php artisan install:api` installs Sanctum and its migration, but does not add the trait to the model.
- The Laravel `users` table also comes from a migration.

Default `User` model:

```
class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;
```

Add one trait to `app/Models/User.php` :

- The `run1-5.sh` script copies a prepared `User.php` that already includes it.

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasFactory, HasApiTokens, Notifiable;
}
```

- `HasApiTokens` gives the user `createToken()`, `tokens()`, and `currentAccessToken()`.

Step 1: Log In and Create a Token

Create the AuthController to enable login.

```
php artisan make:controller Api/AuthController
```

```
namespace App\Http\Controllers\Api;  
  
use App\Http\Controllers\Controller;  
use Illuminate\Http\Request;  
  
class AuthController extends Controller  
{  
    //  
}
```

Add the login method:

```
public function login(Request $request)
{
    $credentials = $request->validate([
        'email' => 'required|email',
        'password' => 'required|string',
    ]);
    $user = User::where('email', $credentials['email'])->first();

    if (! $user || ! Hash::check($credentials['password'], $user->password)) {
        return response()->json(['message' => 'Invalid email or password'], 401);
    }
    $token = $user->createToken('api-token', ['*'], now()->addHour())
        ->plainTextToken;

    return response()->json(['token' => $token, 'token_type' => 'Bearer',
        'expires_in' => 3600]);
}
```

- `Hash::check()` compares the typed password with the stored hash; Passwords are never stored as plain text.
- `createToken(name, abilities, expiresAt)`: this token lasts one hour.
- `plainTextToken` looks like `1|ZDWc...`: the row id, a `|`, and the secret.

- The plain text is shown **only once**. The database keeps just the hash.
- The client must store the returned token securely and send it as `Authorization: Bearer <token>` on protected requests.
- If the client loses it, the token cannot be recovered from the database; request a new one.

```
{"message":"Login successful",  
"token":"1|ZDWcItjHNas6Zi064bMNtxISpKan7h5PsFo8Y2oE8ea98eab",  
"token_type":"Bearer","expires_in":3600,"user":"Student"}%
```

Step 2: Protect the Routes

Define the API routes in `routes/api.php`.

- Routes inside the `auth:sanctum` group require a valid token; routes outside it are public.

```
Route::post('/login', [AuthController::class, 'login']);
Route::get('/students', [StudentController::class, 'index']);

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/me', [AuthController::class, 'me']);
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::post('/students', [StudentController::class, 'store']);
    Route::put('/students/{student}', [StudentController::class, 'update']);
    Route::delete('/students/{student}', [StudentController::class, 'destroy']);
});
```

Route	Token needed?
POST /api/login	No
GET /api/students , GET /api/students/{id}	No
GET /api/me , POST /api/logout	Yes
POST , PUT , DELETE on students	Yes

- `auth:sanctum` is **middleware**: code that runs *before* the controller.
- If the token is missing, wrong, or expired, the middleware stops the request with `401`, and the controller never runs.
- The controller stays clean: it does not contain any token-checking code.

Step 3: Use the Logged-In User

```
public function me(Request $request)
{
    return response()->json(['user' => $request->user()]);
}

public function logout(Request $request)
{
    $request->user()->currentAccessToken()->delete();
    return response()->json(['message' => 'Logged out']);
}
```

- `$request->user()` is the user who owns the token in this request.
- `logout` deletes only this token. The user's other tokens still work.

New Artisan Command Summary

```
php artisan install:api # installs Sanctum
php artisan make:controller Api/AuthController
php artisan migrate --path=database/migrations/<timestamp>_create_personal_access_tokens_table.php
php artisan db:seed --class=UserSeeder
php artisan route:list --path=api
```

```
$user->createToken('name', ['*'], now()->addHour())->plainTextToken;
$request->user();
$request->user()->currentAccessToken()->delete();
```

What Comes Next

- **5-2. run1-5.sh:** how the script builds this project, including the new tables.
- **5-3. Testing Bearer Tokens:** test login, tokens, and logout with `curl` and in the browser.